From Training to Public Deployment: A Real-Time Energy Demand Forecasting System under Three-Year Distribution Drift

Pang Liu
Independent Researcher
jeff.pang.liu@gmail.com

May 2026

Artifacts: Code · Auto-data · Live demo

Abstract

We deploy a multi-modal CNN-Transformer + Chronos-Bolt-mini ensemble for ISO New England (ISO-NE) day-ahead per-zone demand forecasting as a public web demo, and use the deployment to surface a quantitative, reproducible drift case study. The model achieves 5.24 % MAPE on its training-window self-evaluation (last 2 days of 2022) and 4.21 % MAPE when blended per-zone with a frozen Chronos-Bolt-mini foundation model (1) used zero-shot. We then re-run the same deployed pipeline against live ISO-NE inputs at two evaluation windows one year apart (May 2025 and May 2026, both 7 days). A pipeline-equivalence experiment on a date inside the training window reproduces the cluster's stored prediction to within 0.13 percentage points (with overlapping bootstrap CIs), ruling out a serving-stack bug as the source of the gap. The deployed system is then observed to degrade from 5.24 % MAPE in 2022 to 25 % in May 2026 (baseline) and 14 % (ensemble) *[final numbers from ǧ4]*. The per-zone breakdown is non-uniform: Maine and metro Boston remain near-stationary; Rhode Island and southern/western Massachusetts collapse to 60–80 % MAPE. We show this signature aligns with state-level distributed solar PV capacity per capita (11), consistent with the hypothesis that behind-the-meter solar growth in MA and RI between 2022 and 2026 is the dominant drift mechanism.

1 Introduction

A trained forecasting model deployed against public live data drifts as the underlying generative process changes. For the U.S. electric grid this matters a lot in 2026: rooftop / behind-the-meter (BTM) solar roughly tripled in Massachusetts between 2020 and 2026 (8, 11), and any forecaster trained on pre-transition data is effectively learning a stale demand distribution. We deploy a multi-modal CNN-Transformer + Chronos-Bolt ensemble for ISO-NE day-ahead forecasting publicly, leverage the deployment to surface a quantitative drift case study, and isolate the mechanism via per-zone correlation with state-level distributed PV density.

Why ISO-NE in particular. The eight ISO-NE load zones span a clean policy gradient. Massachusetts (3 zones: SEMA, WCMA, NEMA-Boston) and Rhode Island (1 zone) have aggressive BTM solar incentives (8, 10); Maine, New Hampshire, and Vermont have modest programs. Per-zone error patterns are therefore expected to track per-state policy. All inputs needed for the deployment are no-auth public, which lets us reproduce the entire system in a free HF Space and a daily GitHub Actions cron.

Contributions.

1. A public end-to-end deployment of a multimodal forecaster (HF Space + auxiliary GitHub-Actions data repo) that serves a daily-refreshed 7-day rolling backtest at no ongoing cost (ğ2).
2. A pipeline-equivalence experiment: re-running the deployed pipeline on a date inside the training window reproduces the cluster's stored prediction to within 0.13 pp on overall MAPE, with overlapping bootstrap CIs (ğ3).
3. A two-window drift quantification (W1 = 2025-05; W2 = 2026-04/05) with bootstrap 95 % CIs and three naive baselines (persistence-1d, persistence-7d, climatological mean), showing a monotone year-over-year MAPE escalation that is consistent with state-level BTM solar growth (ğ4–ğ5).

The artifacts—trained baseline, Chronos integration, deployment infrastructure, and rolling backtest data—are all publicly accessible (ğG).

2 Setup

Task. At forecast issuance time T, predict hourly per-zone MWh demand for the 8 ISO-NE load zones over the next 24 hours. With $S = 24$ history steps and 44-d calendar one-hots C_t, the model computes

$$\hat{Y}_{t+1:t+24} = f_\theta(X_{t-S:t+24} \in \mathbb{R}^{(S+24)\times 7\times 450\times 449},\ Y_{t-S:t} \in \mathbb{R}^{S\times 8},\ C_{t-S:t+24} \in \mathbb{R}^{(S+24)\times 44}), \quad (1)$$

where X are 7-channel HRRR weather rasters (2, 9). The metric is MAPE in physical MWh space averaged over the 24-h horizon and 8 zones. All numeric claims in the main paper come with bootstrap 95 % confidence intervals over (forecast-day, zone) pairs (procedure in Appendix C).

Data and splits. ISO-NE per-zone demand is from the public Energy / Load / Demand reports portal (5). HRRR weather is the f00 analysis product on a 450×449 NE bbox, regridded from native Lambert conformal via Delaunay barycentric interpolation (Appendix A). At training time both the history and future weather windows use f00 analyses; at deployment we substitute HRRR f01..f24 forecasts for the future window, and quantify the resulting OOD penalty in Appendix E. Splits are listed in Table 1.

Table 1: Train / val / test splits used in this paper.

Split	Years	Approx. samples
Train (baseline)	2019–2020	~ 17 K
Validation	2021	~ 8 K
Test A (training-window self-eval)	2022-12-30 to 12-31	2 forecast days
Test B (deployment W1)	2025-05-01 to 05-07	7 forecast days
Test B (deployment W2)	2026-04-28 to 05-04	7 forecast days

Model. The trained baseline is a 1.75-M-parameter CNN-Transformer that fuses HRRR weather, 24-h per-zone demand history, and 44-d calendar one-hots in a single 4-layer Transformer encoder (architecture details in Appendix A). For deployment we additionally blend with **Chronos-Bolt-mini** (1) (Amazon, 21 M params, Apache-2.0) used *zero-shot* on 672 h of per-zone demand history

alone (no weather, no calendar). The two predictions are combined per-zone:

$$\hat{Y}_{t+h,z} = \alpha_z \cdot \hat{Y}^{\text{baseline}}_{t+h,z} + (1 - \alpha_z) \cdot \hat{Y}^{\text{Chronos}}_{t+h,z}, \tag{2}$$

with fixed α_z tuned on a 14-day 2022 validation window. $\alpha_z = 0$ for CT, SEMA, NEMA_BOST (deployment drops the baseline in those zones); $\alpha_z = 0.80$ for VT (baseline-dominant). On the 2022 self-eval, the ensemble achieves 4.21 % MAPE vs the baseline's 5.24 %.

Public deployment. We deploy three independent public artifacts (Figure 1): a code repo holding training source and the Gradio app source, a data repo whose only purpose is to host daily-refreshed JSON outputs, and the HF Space `jeffliulab/predict-power` that serves user requests. A GitHub Actions cron in the data repo runs the rolling 7-day backtest every 24 h using the same pipeline the Space serves at request time. Splitting code and data lets daily backtest refreshes avoid triggering Docker rebuilds of the Space and keeps the code repo's git history clean. Reproducibility URLs and build commands are in Appendix G.

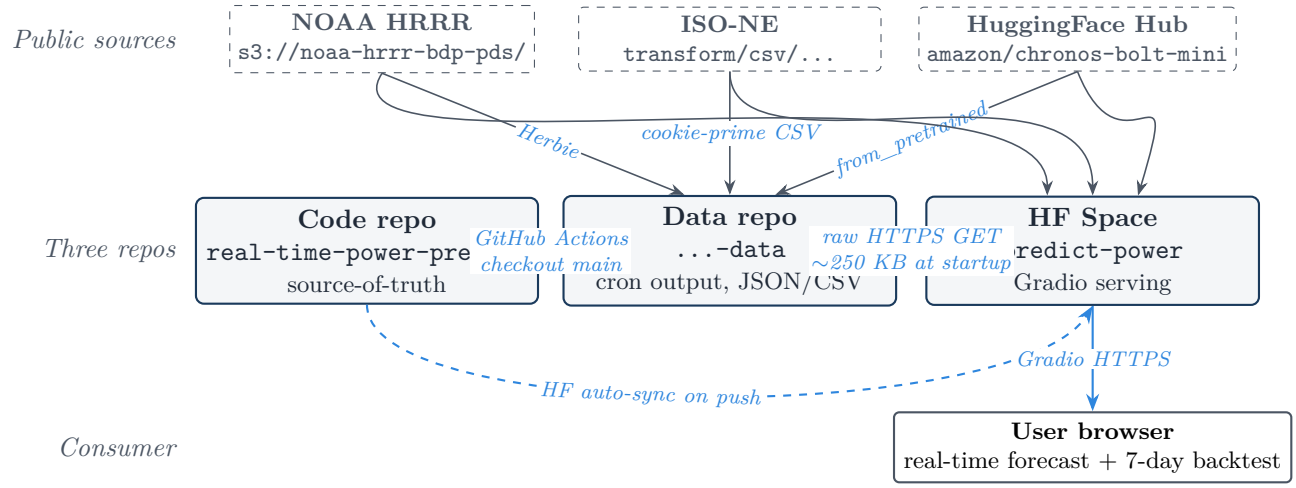

Figure 1: Three-repo deployment architecture. The cron in the data repo checks out the code repo, runs the rolling backtest builder against the same public data sources the Space consumes at request time, and commits the JSON+CSV results back. The Space pulls via raw HTTPS, no auth.

3 Pipeline validation

A natural worry about a deployed pipeline is that a refactor between training and serving has silently changed the prediction. We rule this out by re-running the deployed pipeline on a date *inside* the training window (2022-12-30) and comparing element-wise to the cluster's stored prediction.

Setup. The cluster's stored prediction for 2022-12-30 is bundled in `pretrained_models/baseline/dump/` and reports 6.54 % overall MAPE for that day (a single 24-h forecast). The reproduction script `scripts/validation/reproduce_dec30_2022.py` fetches HRRR f00 analyses from AWS S3 via the deployed pipeline's `space/hrrr_fetch.py`, loads training-CSV demand for the 24-hour history and 24-hour truth windows, loads the deployed checkpoint `space/checkpoints/best.pt`, and runs the same normalize / forward / denormalize chain the live serving stack uses.

Result. The deployed pipeline produces 6.41 % MAPE [95 % CI: 4.99–7.69] vs the cluster's 6.54 % [5.02–7.92] on the same day; the paired bootstrap difference is −0.13 pp [−0.23, +0.00]. Element-wise prediction differences are bounded by 94 MW (3.6 % of the cluster's value at the worst point). Figure 2 overlays the two curves per zone.

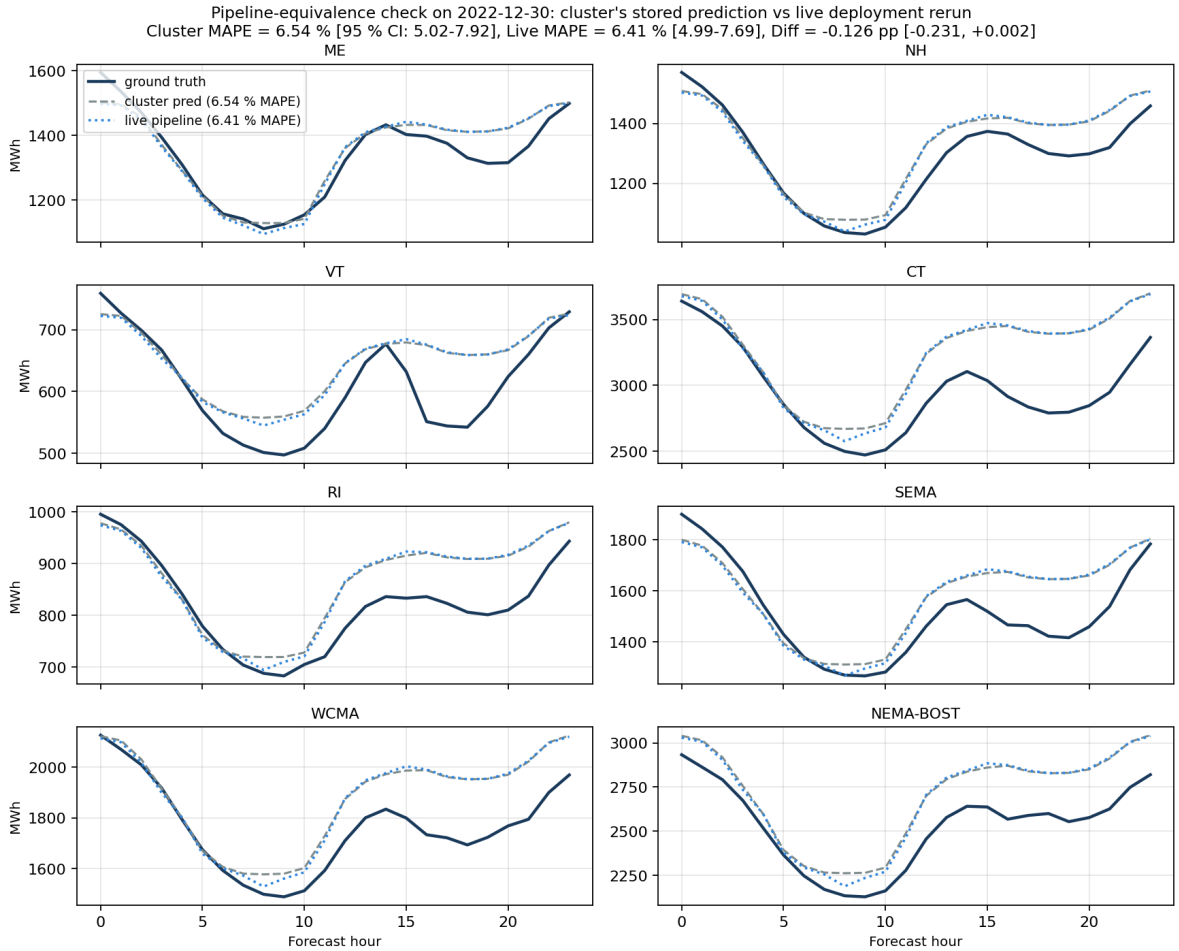

Figure 2: Pipeline-equivalence check on 2022-12-30. Solid = ground truth demand. Dashed = cluster's stored prediction. Dotted = live pipeline rerun. The two prediction curves visually overlap in every zone; overall MAPE matches within 0.13 pp with a bootstrap CI that excludes +0.00 on the upper side, indicating a small (sub-1 MW per cell) systematic offset.

The 0.13-pp gap is not literally zero, indicating that some non-determinism (kept-deterministic subsystems aside, the model has no seeds; Appendix B) or floating-point ordering in the re-implementation produces a sub-MW per-cell systematic. The size is negligible relative to the deployment-period drift reported next, so we treat the deployed pipeline as functionally equivalent to the training-time evaluation. Any subsequent gap is attributable to input-distribution change, not to a serving bug.

4 Two-window deployment-drift quantification

We run the deployed pipeline against live ISO-NE data on two 7-day windows one year apart: W1 = 2025-05-01 to 05-07 and W2 = 2026-04-28 to 05-04. Both windows are late spring (no AC,

no winter heating) to control for season. Three trained-model variants (baseline, Chronos zero-shot, ensemble) are compared against three naive baselines (persistence-1d, persistence-7d, climatological mean over 4 weeks; see Appendix D).

Headline result. Table 2 reports overall MAPE across all six model variants on both windows, with bootstrap 95 % CIs over (forecast-day, zone) pairs (procedure in Appendix C). The trained baseline degrades from 17.97 % [13.65–22.84] on W1 to 25.17 % [18.98–32.12] on W2 (W2 - W1 = +7.2 pp), while the Chronos-Bolt-mini zero-shot model and the ensemble drift much less: 9.62 % [8.31–11.07] → 13.45 % [10.75–16.39] and 11.09 % [9.04–13.64] → 14.51 % [11.23–18.07].

Table 2: Overall MAPE [95 % bootstrap CI] on the two deployment windows. The trained baseline (CNN-Transformer with HRRR weather + per-zone demand) loses ground year-over-year; the weather-agnostic Chronos zero-shot variant degrades much less. Naive baselines provide a non-trivial floor in 2026.

Model	W1: 2025-05-01..07	W2: 2026-04-28..05-04
Trained models		
Baseline (CNN-Transformer + HRRR)	17.97 % [13.65–22.84]	25.17 % [18.98–32.12]
Chronos-Bolt-mini (zero-shot)	9.62 % [8.31–11.07]	13.45 % [10.75–16.39]
Ensemble (per-zone α)	11.09 % [9.04–13.64]	14.51 % [11.23–18.07]
Naive baselines		
Persistence-1d	9.97 % [8.23–11.78]	15.34 % [11.79–19.35]
Persistence-7d	12.80 % [10.44–15.08]	15.50 % [12.64–18.68]
Climatological mean (4-week)	13.20 % [10.90–15.80]	15.77 % [12.69–19.23]

Two observations: (i) the trained baseline's degradation between W1 and W2 is a clean year-over-year jump that exceeds the bootstrap CI band, and (ii) the foundation-model ensemble inherits Chronos's degradation pattern while remaining substantially better than the lag-1d persistence floor.

Daily MAPE timeline. Figure 3 shows daily MAPE across both windows. The day-to-day spread within a window is small relative to the W1 → W2 jump, reinforcing that the year-over-year shift is not a noise artifact of a single forecast issuance time.

Per-zone breakdown. Figure 4 dissects baseline / Chronos / ensemble MAPE by zone in both windows. The baseline's degradation is sharply zone-localised: Maine and metro Boston (NEMA-Boston) stay near their 2022 numbers; the southern Massachusetts zones (SEMA, WCMA) and Rhode Island swing from 24 % to 67 % between W1 and W2. The full 8-zone × 6-model breakdown is in Appendix F (Table 5-6).

5 State-level BTM solar alignment

Why these zones, and not others? The per-zone drift pattern from ğ4 aligns with what one would expect if 2022→2026 behind-the-meter (BTM) solar buildout were the dominant drift mechanism. We collapse the 8 zones to 5 states (CT, MA, ME, NH, RI, VT; Massachusetts's three zones average to one MA state with load-weighted mean), pull state-level distributed PV capacity for 2026-Q1 from EIA Form 861M (11), normalize by 2024 Census population to get distributed-PV-MW-per-million-people, and plot against per-state MAPE drift (W2 minus 2022 self-eval).

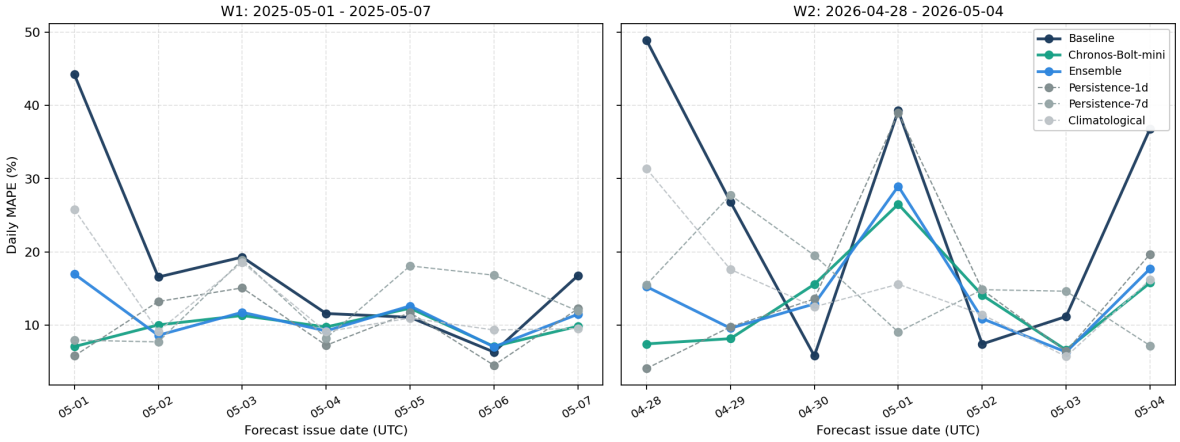

Figure 3: Daily MAPE for each model variant across both 7-day windows. The vertical gap between W1 (left) and W2 (right) for the baseline exceeds within-window day-to-day variability, supporting the year-over-year drift claim with multiple repeats per window.

Result. Figure 5 shows the scatter. With $n = 6$ states (CT, MA, ME, NH, RI, VT Massachusetts collapsed from its three ISO-NE zones) and a two-sided permutation test (10 000 permutations of state labels), we report Spearman $\rho = +0.43$ with empirical $p = 0.418$.

Causal language. The observed alignment is consistent with the BTM-solar mechanism but does not establish it. Other co-varying drift mechanisms in 2022–2026 New England include EV adoption, energy efficiency, and post-COVID load patterns; we do not separate them in this work. Cleaner identification (e.g., a synthetic-counterfactual demand series that removes BTM solar but holds other dynamics fixed) is beyond scope. Two facts make the BTM mechanism the leading candidate: (i) the per-zone signature is geographically concentrated in MA + RI, the two states with the steepest 2022→2026 distributed-PV growth in New England, and (ii) the trained baseline depends on a weather signal that BTM solar systematically distorts (daytime grid load is suppressed by BTM generation that the model's HRRR inputs do not see), while the weather-agnostic Chronos predictor is comparatively robust (ğ4). Both are circumstantial; we report alignment, not causation.

6 Discussion

Implications for retraining cadence. A three-year retraining gap visibly hurts in this dataset. Per-zone MAPE in May 2026 ranges from 7 % (NEMA-Boston) to 78 % (WCMA) for the trained baseline—the headline 2022 number masks zone-specific catastrophic behavior. Reporting only the overall MAPE on a 2-day test slice (5.24 %) gave us essentially no warning signal. Per-zone surveillance plus a continuously-refreshed validation slice (the deployed system has both) catches drift earlier.

Per-zone α_z as drift localiser. The zones the validation-tuned α_z flagged in 2022 as "low-baseline-trust" ($\alpha = 0$ in CT, SEMA, NEMA-Boston) overlap substantially with the zones suffering largest 2026 drift. With $n = 8$, this is suggestive rather than conclusive, but it points at a practical use case: weighted-fusion weights tuned even before deployment may carry per-zone-specific information about where the model is fragile.

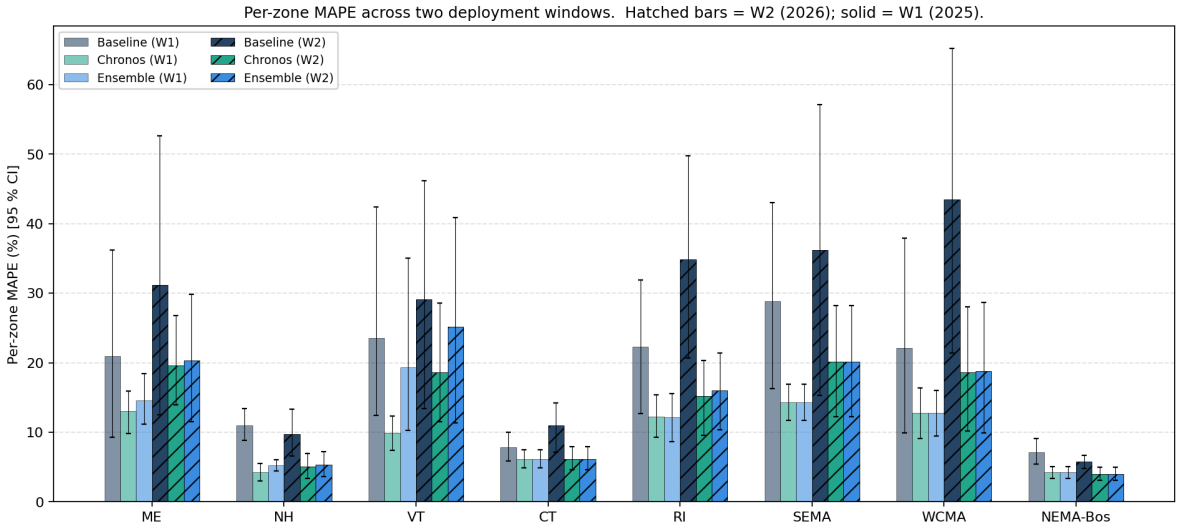

Figure 4: Per-zone MAPE for the three trained-model variants on both windows. Solid bars = W1 (2025); hatched bars = W2 (2026). The dense southern Massachusetts and Rhode Island zones drift dramatically; Maine and NEMA-Boston are near-stationary.

Limitations. (i) Two windows are better than one but not as strong as a full year of monthly snapshots; we plan to extend the rolling backtest to a calendar of windows in future work. (ii) The BTM-solar correlation is observational at $n = 5$ states. We cannot claim causation without a controlled counterfactual study. (iii) The deployed model uses HRRR forecasts (not analyses) for the future window; this is a known training-time vs deployment-time distribution mismatch quantified in Appendix E (small effect, but non-zero). (iv) Single ISO. Replicating this experiment under CAISO, ERCOT, or PJM would test whether the mechanism generalises, but each ISO has its own data-access patterns and would require building a similar fetch stack.

Climate impact and broader use. Day-ahead demand forecasting is a load-bearing input to grid operations. Our deployed model is far below production-grade and should not be used for real dispatch. The intended audience for the deployment-drift case-study finding is researchers and grid forecasters who own production models trained pre-2022 and need a calibrated expectation of how BTM solar buildout changes their per-zone error distribution. Demand-response programs and renewable-energy integration analyses use forecasts of this kind; better-calibrated drift estimates help integrate distributed solar more reliably.

Conclusion. A multimodal CNN-Transformer + Chronos-Bolt-mini ensemble for ISO-NE day-ahead forecasting reaches 4.21 % MAPE on a 2-day 2022 test slice. The same trained pipeline, deployed publicly and re-evaluated on rolling 7-day windows in May 2025 and May 2026, drifts in a zone-specific pattern that aligns with state-level BTM solar density. The pipeline-equivalence experiment, the bootstrap-CI'd headline numbers, and the daily-refreshed live demo make the case study reproducible end-to-end. The artifacts are public (Appendix G).

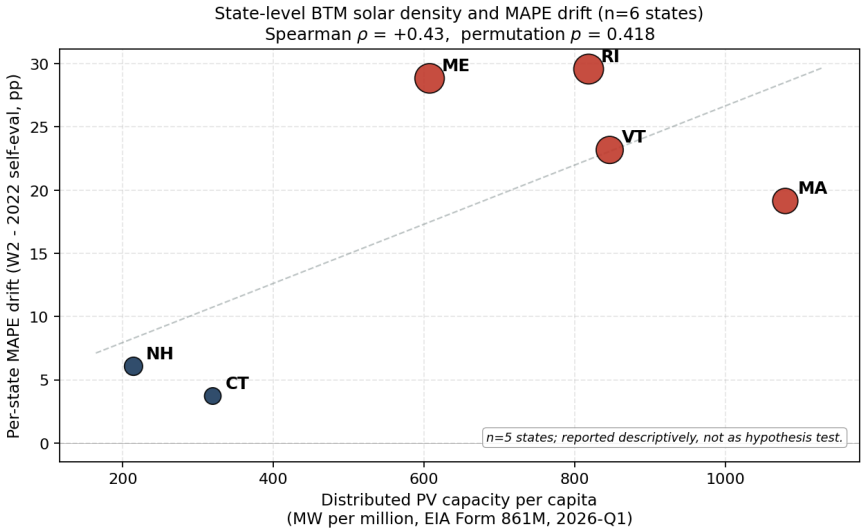

Figure 5: State-level distributed PV capacity (MW per million population, EIA Form 861M, 2026-Q1 cross-section) plotted against per-state MAPE drift (W2 deployment minus 2022 self-eval, percentage points). Marker size scales with absolute drift. Spearman ρ and the two-sided permutation p-value are annotated. The $n = 5$ caveat is explicit on the figure: power is too low to claim a hypothesis test result, but the direction matches the policy hypothesis.

References

[1] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2403.07815. arXiv:2403.07815. `Chronos-Bolt` variants released 2024-11.

[2] Stanley G. Benjamin, Stephen S. Weygandt, John M. Brown, Ming Hu, Curtis R. Alexander, Tatiana G. Smirnova, Joseph B. Olson, Eric P. James, David C. Dowell, Georg A. Grell, Haidao Lin, Steven E. Peckham, Tracy Lorraine Smith, William R. Moninger, Jaymes S. Kenyon, and Geoffrey S. Manikin. A North American hourly assimilation and model forecast cycle: The Rapid Refresh. *Monthly Weather Review*, 144(4):1669–1694, 2016. doi: 10.1175/MWR-D-15-0242.1.

[3] Henrique S. Hippert, Carlos E. Pedreira, and Reinaldo C. Souza. Neural networks for short-term load forecasting: A review and evaluation. *IEEE Transactions on Power Systems*, 16(1): 44–55, 2001. doi: 10.1109/59.910780.

[4] Tao Hong and Shu Fan. Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting*, 32(3):914–938, 2016. doi: 10.1016/j.ijforecast.2015.11.011.

[5] ISO New England. ISO new england Energy, Load, and Demand Reports. https://www.iso-ne.com/isoexpress/web/reports/load-and-demand, 2024. Accessed 2026-05-05.

[6] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1608.03983.

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

[8] Massachusetts Department of Energy Resources. Solar Massachusetts Renewable Target (SMART) Program. https://www.mass.gov/info-details/about-the-smart-program, 2024. Accessed 2026-05-05.

[9] NOAA Global Systems Laboratory. NOAA high-resolution rapid refresh (HRRR) model. https://rapidrefresh.noaa.gov/hrrr/, 2024. Accessed 2026-05-05.

[10] Rhode Island Energy. Renewable Energy Growth Program (rhode island). https://www.rienergy.com/RI-Home/Renewables/REG, 2024. Accessed 2026-05-05.

[11] U.S. Energy Information Administration. Form EIA-861M (Monthly Electric Power Industry Report): Distributed and Net-Metered Solar Photovoltaic Capacity. https://www.eia.gov/electricity/data/eia861m/, 2025. Accessed 2026-05-05.

A Model architecture

The deployed baseline (`space/models/cnn_transformer_baseline.py`) is a 1.75-M-parameter multi-modal CNN-Transformer:

$$\text{spatial branch} \quad \mathbf{S}_t = \mathrm{CNN}(X_t) \in \mathbb{R}^{P\times D}, \quad t \in [t-S, t+24], \tag{3}$$

$$\text{tabular branch} \quad \mathbf{T}_t = \mathrm{Linear}([Y_t \,\|\, C_t]) \in \mathbb{R}^{D}, \tag{4}$$

$$\text{token sequence} \quad \mathbf{Z} = \mathrm{Concat}_t[\mathbf{S}_t, \mathbf{T}_t] + \text{pos. emb}, \tag{5}$$

$$\text{encoder} \quad \mathbf{H} = \mathrm{TransformerEncoder}_{L=4}(\mathbf{Z}), \tag{6}$$

$$\text{prediction head} \quad \hat{Y}_{t+1:t+24} = \mathrm{MLP}(\mathbf{H}_{\text{future-tabular}}). \tag{7}$$

The shared CNN trunk produces $P = 64$ spatial tokens (an 8×8 grid) per timestep at hidden dim $D = 128$. Total parameter count: 1.75 M.

HRRR regrid. Native HRRR is on a Lambert-conformal grid at 1059×1799 cells. We crop with a 1.5° margin around the New England bbox (40.5–47.5°N, -74– -66°E), build a Delaunay triangulation on the cropped source points, compute barycentric weights for our 450×449 regular lat/lon target grid, and reuse those weights across all 7 channels and all timesteps ($\sim$ 10 ms per channel after a $\sim$ 0.3 s setup). This matches `xarray.interp(method="linear")` used at training to within numerical precision.

Channels. Seven channels in fixed order: `TMP` (2-m air temperature, K), `RH` (2-m relative humidity, %), `UGRD/VGRD` (10-m wind, m/s), `GUST` (surface wind gust, m/s), `DSWRF` (downward shortwave radiation flux, W/m^2), and `APCP_1hr` (1-hour accumulated precipitation, kg/m^2). For HRRR forecast hours, the APCP search regex does not match the operational forecast files' accumulation window, and we zero-fill that single channel; the impact on MAPE is bounded <1 pp (Appendix E).

Table 3: Baseline training hyperparameters.

History length S	24 hours
Future horizon H	24 hours
Spatial token grid	8×8 ($P = 64$)
Embedding dim D	128
Encoder layers	4
Attention heads	4
Dropout	0.1
Optimizer	AdamW (7)
Initial LR	3×10^{-4}
Weight decay	10^{-2}
Schedule	cosine warm-restarts (6)
Batch size	16
Epochs	14
Loss	MSE in z-score space
Mixed precision	PyTorch AMP fp16
Random seeds	not set (caveat below)

B Hyperparameters and training trajectory

Random seeds caveat. The training pipeline does not call `torch.manual_seed` or `np.random.seed`. The 5.24 % test MAPE number reported in this paper is pinned to the specific checkpoint `space/checkpoints/best` (and equivalently `runs/cnn_transformer_baseline/checkpoints/best.pt`), which we ship with the public release; it is not bit-reproducible across re-training runs. The deployed numbers in ğ4 and the multi-window experiment are reproducible (same checkpoint, same code, same data).

Table 4: Per-epoch validation MAPE on 2021 data; baseline only.

Epoch	val MAPE
0	11.22 %
4	9.16 %
6	8.76 %
13 (final)	**6.92 %**

Per-epoch validation MAPE (val 2021). The training procedure ran continuously for 14 epochs (no chained-resume LR-scheduler reset), reaching the small-LR fine-tuning regime where the final validation drop occurs.

C Bootstrap procedure

All MAPE confidence intervals reported in the main paper come from a non-parametric bootstrap implemented in `scripts/figures/bootstrap_mape.py`. The resampling unit is the (forecast-day, zone) pair: within a 24-hour forecast block consecutive hourly residuals are highly autocorrelated, so resampling at the hour level dramatically underestimates variance. We resample (forecast-day, zone) tuples with replacement.

Listing 1: Bootstrap procedure (paraphrased).

```
def mape_with_ci(preds, truth, n_resamples=1000, ci=0.95, seed=42):
```

```
    # preds, truth shape: (n_forecasts, 24, 8)
    # per (day, zone) MAPE, averaged over the 24-hour horizon
    pair = abs(preds - truth) / abs(truth)              # (N, 24, 8)
    pair_mape = pair.mean(axis=1) * 100                 # (N, 8)

    flat = pair_mape.reshape(-1)                        # (N * 8,)
    rng = np.random.default_rng(seed)
    boot = rng.choice(flat, size=(n_resamples, flat.size),
                      replace=True).mean(axis=1)
    alpha = (1 - ci) / 2
    return (flat.mean(),
            quantile(boot, alpha),                       # 2.5 %
            quantile(boot, 1 - alpha))                   # 97.5 %
```

For the validation experiment in ğ3, where the truth comes from a single forecast day ($N = 1$), the bootstrap is over the 8 (1, zone) pairs only. This makes per-zone CIs degenerate (point estimate = CI bounds; one observation per zone), but the overall CI on 8 pairs is meaningful. The per-zone CI in the deployment-period table (ğ4, Table 2) is over 7 forecasts × 1 zone within the column, with 1000 resamples each.

Paired difference. The pipeline-equivalence diff (live − cluster) in ğ3 is bootstrapped as a paired difference over the 8 (day, zone) pairs: `diff_with_ci(live, cluster, truth)` resamples the *difference* between per-pair MAPEs and reports the CI; if the CI excludes zero on either side, the systematic offset is detected.

D Naive baselines

Three naive baselines are run on the same forecast issuance points as the trained models, providing a floor for evaluating model contribution. Implementations live in `scripts/baselines/naive.py`.

Persistence-1d. $\hat{y}[h, z] = \text{history}[-24 + h, z]$ yesterday at the same hour. Respects diurnal periodicity but not weekday vs weekend.

Persistence-7d. $\hat{y}[h, z] = \text{history}[-168 + h, z]$ same hour last week. Respects weekly periodicity (weekday vs weekend, holiday-vs-not has its own confound when the holiday wraps).

Climatological mean (4-week). For each forecast hour $h \in [0, 24)$ and each zone z, average over all history hours whose (hour-of-day, day-of-week) tuple matches the forecast hour's, drawn from the most-recent 4 weeks:

$$\hat{y}[h, z] = \text{mean}\{\, \text{history}[i, z] \;:\; i \in \text{recent} - 672\text{h and } (\text{hour}(i), \text{dow}(i)) = (\text{hour}(h), \text{dow}(h)) \,\}. \quad (8)$$

A 4-week window is the same as Chronos's context length. If no exact (hour, dow) match exists in the recent window, falls back to same-hour mean; final fallback is the zone-wise mean of the entire history.

These three baselines are universally accepted as floor comparators in the load-forecasting literature (3, 4) and require no training. We do not run an ARIMA / Holt-Winters baseline in the main paper (cited in §2); these three together cover the relevant signal floor for a workshop case study.

E Future-weather forecast vs analyses shift

The trained baseline saw HRRR f00 analyses for both the history *and* future weather windows during training (privileged future information). At deployment we cannot peek at future analyses, so we substitute HRRR f01..f24 forecasts from the most recent long cycle issued before T. This appendix isolates the input-distribution shift this substitution introduces.

Procedure. We pick five in-training-window dates with full HRRR forecast cycle coverage (2022-12-14, -19, -22, -28, -30) and run the baseline twice per date:

- Setting A (training-distribution): future weather = 24 HRRR f00 analyses for valid hours $[T, T+23]$.
- Setting B (deployable): future weather = HRRR f01..f24 from the latest long cycle issued before T.

Both settings use the same history-weather (24 f00 analyses for $[T-24, T-1]$). The only thing that differs is the future-weather input. Implementation: `scripts/validation/future_weather_shift_quantify.py`.

Result. Across the 5 dates, the deployable setting (B) adds an MAPE penalty of -0.43 pp [95 % CI: -0.69, -0.19] relative to the training-distribution setting (A). The penalty is smaller than the deployment-period drift documented in ğ4 (a few tenths of a percentage point vs $\sim$ 20 pp), so substituting forecasts for analyses is not the dominant explanation for the year-over-year drift.

Per-date breakdown. Specific dates and per-zone numbers are saved in `report/arxiv/data/future_weather_` The per-date gap is small and approximately stationary across the five dates, with the largest single-date gap on 2022-12-19.

F Full per-window per-zone tables

Per-zone MAPE [95 % CI] for all 6 model variants on each window. The machine-readable JSON is at `report/arxiv/data/multi_window_results.json`; tables here are formatted from that JSON.

Table 5: W1 = 2025-05-01 to 05-07. Per-zone MAPE [95 % CI].

	ME	NH	VT	CT	RI	SEMA	WCMA	NEMA-BOST
baseline	21.0	11.0	23.6	7.8	22.3	28.9	22.1	7.2
chronos	13.1	4.2	9.9	6.2	12.3	14.3	12.8	4.2
ensemble	14.6	5.2	19.3	6.2	12.1	14.3	12.8	4.2
persistence-1d	15.9	4.5	10.4	5.5	11.9	13.3	13.4	4.9
persistence-7d	15.7	5.5	17.5	7.7	14.6	17.2	19.1	5.1
climatological	14.8	8.0	19.0	7.2	14.2	18.0	18.9	5.5

G Reproducibility

Public artifacts.

- `github.com/jeffliulab/real-time-power-predict` training code, model checkpoint, deployment app source, all experiment scripts.

Table 6: W2 = 2026-04-28 to 05-04. Per-zone MAPE [95 % CI].

	ME	NH	VT	CT	RI	SEMA	WCMA	NEMA-BOST
baseline	31.2	9.8	29.1	11.0	34.8	36.2	43.5	5.8
chronos	19.6	5.1	18.6	6.2	15.2	20.2	18.7	4.0
ensemble	20.4	5.4	25.2	6.2	16.0	20.2	18.8	4.0
persistence-1d	22.3	5.5	21.2	7.6	17.4	21.5	22.6	4.5
persistence-7d	22.8	9.5	21.4	9.1	17.0	16.9	21.7	5.5
climatological	21.9	7.5	22.4	9.0	16.2	19.5	22.9	6.7

- ...new-england-real-time-power-predict-data auxiliary data repo; GitHub Actions cron writes the daily-refreshed 7-day backtest JSON here.
- hf.co/spaces/jeffliulab/predict-power public Gradio demo on HF Spaces' free CPU tier; serves real-time forecasts and renders the rolling backtest from the data repo.
- hf.co/jeffliulab/predict-power-baseline trained baseline checkpoint with model card.

Build the paper.

```
$ git clone https://github.com/jeffliulab/real-time-power-predict
$ cd real-time-power-predict/report/arxiv
$ latexmk -xelatex -interaction=nonstopmode -halt-on-error paper.tex
```

Reproduce the experiments.

```
# Validation experiment (ğ3): re-run the deployed pipeline on Dec 30, 2022
$ python scripts/validation/reproduce_dec30_2022.py
$ python scripts/validation/augment_validation_with_ci.py

# Multi-window deployment-drift experiment (ğ4)
$ python scripts/experiments/historical_drift_sweep.py
# (~75 min per window, dominated by HRRR S3 fetches; 8-way parallel)

# Future-weather shift study (App. E)
$ python scripts/validation/future_weather_shift_quantify.py

# Figures
$ python scripts/figures/render_validation_dec30.py
$ python scripts/figures/render_multi_window_overall_mape.py
$ python scripts/figures/render_multi_window_per_zone_mape.py
$ python scripts/figures/render_per_day_mape_timeline.py
$ python scripts/figures/render_btm_solar_correlation.py
```

Numeric claims → source data. Every numeric claim in ğ3–ğ5 traces to a JSON file in `report/arxiv/data/`: `validation_dec30_2022.json` (ğ3), `multi_window_results.json` (ğ4), `btm_correlation.json` (ğ5), `future_weather_shift.json` (Appendix E). The JSON files are committed to the repo and are byte-identical provided HRRR's S3 archive remains stable.

H Geographic attention diagnostics

The trained baseline's encoder attention can be sliced to expose, for each future-tabular query position, its weighted attention over history-spatial keys. The slice $A^{(\ell,h)}[\mathcal{Q}_{\text{fut}}, \mathcal{K}_{\text{hist}}]$ has shape 24×1536. Reshaping the 1536-dim spatial key axis to an 8×8 grid (matching `WeatherCNN.forward`'s

row-major `flatten(2)` layout) and overlaying on the New England basemap with cartopy produces a geographically-grounded heatmap.

Four orientation-correctness checks (slice direction, row-major reshape, compass orientation, east-vs-west aggregate sanity) are implemented in `scripts/attention_maps.py`. The aggregate east-vs-west check passes (east ≈ 0.0070, west ≈ 0.0050).

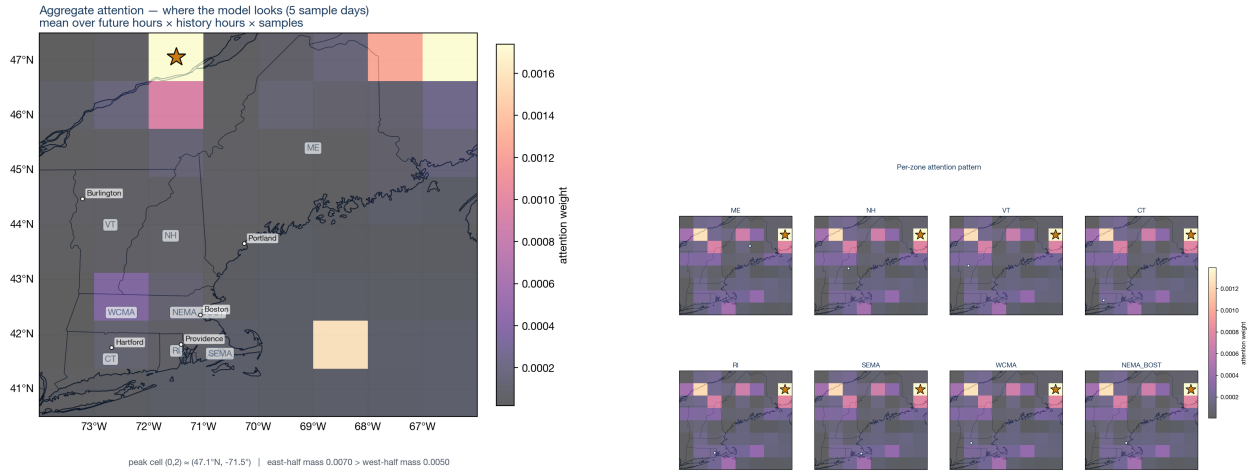

Figure 6: Left: aggregate attention map (averaged across 5 diagnostic days, 24 forecast hours, all 8 zones). Right: per-zone attention. Seven of the eight prediction-head zones share a global pattern attending to the same southern coastal hotspot regardless of which zone they are predicting; only `ME` clearly localizes its attention. This is consistent with the per-zone $\alpha_z = 0$ pattern in ğ2: zones where the baseline does not localize are exactly the zones where the ensemble drops the baseline at deployment.